

Bangladesh University of Engineering & Technology  
Department of Computer Science & Engineering  
CSE 310 - Compiler Sessional

January 2026

## Practice Assignment 2 on Lexical Analysis

### 1 Introduction

In this assignment, we are going to construct a **lexical analyzer**. Lexical analysis is the process of scanning the source program as a sequence of characters and converting them into sequences of tokens. A program that performs this task is called a **lexical analyzer** or a **lexer** or a **scanner**. For example, if a portion of the source program contains `int x=5;` the scanner would convert in a sequence of tokens like:

`<INT> <ID,x> <ASSIGNOP,=> <CONST_NUM,5> <SEMICOLON>.`

After successfully completing the construction of a simple symbol table, we will construct a scanner for a subset of C language. The task will be performed using a tool named **Flex (Fast Lexical Analyzer Generator)** which is a popular tool for generating scanners.

### 2 Tasks

You have to perform the following tasks in this assignment.

#### 2.1 Identifying Tokens

##### 2.1.1 Keywords

You have to identify the keywords given in Table 1 and print the token in the output file. For example, you will have to print `<IF>` in case you find the keyword `"if"` in the source program. **Keywords will not be inserted in the symbol table.**

| Keyword | Token   | Keyword  | Token    | Keyword  | Token    |
|---------|---------|----------|----------|----------|----------|
| if      | IF      | else     | ELSE     | goto     | GOTO     |
| for     | FOR     | while    | WHILE    | long     | LONG     |
| do      | DO      | break    | BREAK    | short    | SHORT    |
| int     | INT     | char     | CHAR     | static   | STATIC   |
| float   | FLOAT   | double   | DOUBLE   | unsigned | UNSIGNED |
| void    | VOID    | return   | RETURN   |          |          |
| switch  | SWITCH  | case     | CASE     |          |          |
| default | DEFAULT | continue | CONTINUE |          |          |

Table 1 : Keyword List

## 2.1.2 Constants

For each constant you have to print a token of the format **<Type, Symbol>** in the output file and **insert the symbol in the symbol table**.

- ❑ **Integer Literals:** One or more consecutive digits form an integer literal. Type of token will be **CONST\_INT**. Note that + or - will not be the part of an integer.
- ❑ **Floating Point Literals:** Numbers like 3.14159, 3.14159E-10, .314159, 000.0314, 314159E10 will be considered as floating point constants. In this case, token type will be **CONST\_FLOAT**.
- ❑ **Character Literals:** Character literals are **enclosed within single quotes**. There will be a single character within the single quotes with the exception of **'\n', '\t', '\\', '\'', '\a', '\f', '\r', '\b', '\v' and '\0'**. For character literals token type will be **CONST\_CHAR**.

Note that, you need to convert the detected lexeme to the actual character. For example if you find **'a'**, then you need to print **<CONST\_CHAR, a>**. That means we only need the **ASCII code**, not the quote symbols around it. Similarly, **you need a newline character (ASCII code 10) in your token if you detect '\n'**.

## 2.1.3 Operators and Punctuators

The operator list for the subset of the C language we are dealing with is given in Table 2. A token in the form of **<Type, Symbol>** should be printed in the output token file. You do not

need to insert the operators and punctuators in the symbol table. Note that if you find LCURL or RCURL , you need to enter a new scope or exit the current scope respectively.

| Symbols              | Type      |
|----------------------|-----------|
| +, -                 | ADDOP     |
| *, /, %              | MULOP     |
| ++, - -              | INCOP     |
| <, <=, >, >=, ==, != | RELOP     |
| =                    | ASSIGNOP  |
| &&,                  | LOGICOP   |
| !                    | NOT       |
| (                    | LPAREN    |
| )                    | RPAREN    |
| {                    | LCURL     |
| }                    | RCURL     |
| [                    | LTHIRD    |
| ]                    | RTHIRD    |
| ,                    | COMMA     |
| ;                    | SEMICOLON |

Table 2: Operators and Punctuators List

## 2.1.4 Identifiers

Identifiers are names given to C entities, such as variables, functions, structures etc. An identifier can only have alphanumeric characters (a-z, A-Z, 0-9) and underscore (\_). The first character of an identifier can only contain alphabet (a-z, A-Z) or underscore (\_). For any identifier encountered in the input file you have to print the token **<ID, Symbol>** and also insert it in the symbol table.

## 2.1.5 Strings

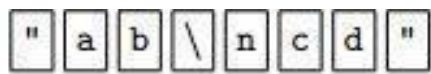
String literals or constants are enclosed in double quotes ". String can be single line or multi line. A multi line string is ended with a \ character in each line except the last line. You have to print a token like **<STRING, abc>** if you find a string "abc" in the input file. Strings will not be inserted in the symbol table.

---

```
"This is a single line string";  
"This is a\  
multiline\  
string"
```

---

Note that, just like character literals, you need to convert the special characters to their original value. If a string contains a `\n`, then you need to replace that two characters with a newline character. For example if the source program contains this eight characters:



|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| " | a | b | \ | n | c | d | " |
|---|---|---|---|---|---|---|---|

Then the scanner should convert it into the following five characters:



|   |   |    |   |   |
|---|---|----|---|---|
| a | b | \n | c | d |
|---|---|----|---|---|

### 2.1.6 Comments

Comments can be single lined or multiple lined. A single line comment usually starts with `//` symbols. However a comment started with `//` can be continued to the next line if the first line ends with a `\`. A multiline comment starts with `/*` and terminate with the characters `*/`. If there is any comment in the input file you have to recognize it but not generate any token in the output file.

---

```
// A single line comment  
// A multiple line\  
comment  
/** Another multiple  
line Comment */
```

---

### 2.1.7 White Space

You have to capture the white spaces in the input file, but no actions needed regarding this.

## 2.2 Line Count

You should count the number of lines in the source program.

## 2.3 Lexical Errors

You should detect lexical errors in the source program and report it along with corresponding line no. . You have to detect the following type of errors:

- ☐ Too many decimal point error for character sequence like **1.2.345.789**
- ☐ Ill formed number such as **1E10.7**
- ☐ Invalid Suffix on numeric constant or invalid prefix on identifier for character sequence like **12abcd**
- ☐ Multi character constant error for character sequence like **'abc'**
- ☐ Unfinished character such as **'a' , '\n' or '\'**
- ☐ Empty character constant error
- ☐ Unfinished string
- ☐ Unfinished comment
- ☐ Unrecognized character

**Detecting errors of the classes mentioned is enough. You don't have to think about other errors for now.** Also count the total number of errors.

## 3 Input

The input will be a text file containing a C source program. File name will be given from the command line.

## 4 Output

In this assignment, there will be two output files. One is a file containing tokens. This file should be named as <YourStudentID>\_token (For example 2105200\_token.txt). You will output all the tokens in this file.

The other file is a log file named as <YourStudentID>\_log.txt. In this file you will output all the actions performed in your program. For example, after detecting any lexeme except one representing white spaces you will print a line containing

**Line No. <line\_count>: Token <Token> Lexeme <Lexeme> found.**

For example, if you find a comment `//abcd` at line no. 5 in your source code you will print  
**Line No. 5: Token COMMENT Lexeme //abcd found.**

Note that, although you will not print any token in the corresponding token.txt file for comments, you will print it in the log file. **For any insertion into the symbol table, you will print the symbol table in the output log file** (only print the non empty buckets). If a symbol already exists, print an appropriate message. For any detected error print

**Line No. 5:<Corresponding error message>.**

Print the line count and total number of errors found at the end of the log file. Also in the case of **CONST\_CHAR** and **STRING** tokens, you have to print the corresponding token found in the log file. For more clarification about input output please refer to the sample input output file given in Moodle. You are highly encouraged to produce output exactly like the sample one. For matching the output files, please use the following hash function with **bucket size 7**.

```
unsigned int sdbmHash(const char *p) {
    unsigned int hash = 0;
    auto *str = (unsigned char *) p;
    int c{};
    while ((c = *str++)) {
        hash = c + (hash << 6) + (hash << 16) - hash;
    }
    return hash;
}
```

## 5 Submission

This is a practice assignment. But you have to make a submission via moodle, which will ensure you rewards at the end of the term. Please follow the steps given below to submit your assignment.

1. In your local machine create a new folder whose name is your 7 digit student id.
2. Put the lex file named as `<your_student_id>.l` containing your code. Also put any additional C file or header file that is necessary to compile your lex file. **Do not put the generated lex.yy.c file or any executable file in this folder.**
3. Compress the folder in a .zip file which should be named as your 7 digit student id.
4. Submit the .zip file.

## 6 Deadline

The deadline is set on **Friday, May 24, 2026 at 11:55 PM.**

For any query, contact: [rumi@teacher.cse.buet.ac.bd](mailto:rumi@teacher.cse.buet.ac.bd)